

PA 1: Implementing xv6 User Commands and System Calls

Task 1. (5 pts) Implement the uptime utility

I created my program inside user/uptime.c and added it to UPROGS in the Makefile so it builds into the xv6 shell. My program calls the already provided uptime() system call and prints out how many clock ticks have passed since xv6 booted up.

Output:

```
$ make qemu
...
init: starting sh
$ uptime
up 24 clock ticks
```

Task 2. (10 pts) Implement a time1 command that reports elapsed time.

I wrote my code in user/time1.c and added it to the Makefile. It first checks if the user typed in a command to run. Then it calls uptime() to get the start time, uses fork() to create a child process, and uses exec() to run the command inside that child. The parent process calls wait() until the child is done and then calls uptime() again to see the end time. Subtracting the start time from the end time gives the total elapsed ticks.

Output:

```
$ time1 sleep 10
elapsed time: 10 ticks
$ time1 matmul
Time: 40 ticks
elapsed time: 40 ticks
```

Task 3. (10 pts) Keep track of how much cputime a process has used.

To keep track of how long each process actually uses the CPU, I edited three files:

- **kernel/proc.h:** Added uint cputime; to struct proc so every process has its own counter variable.
- **kernel/proc.c:** Inside allocproc(), I set p->cputime = 0; so that whenever a brand-new process is created, its counter starts at zero.
- **kernel/trap.c:** In both usertrap() and kerneltrap(), there is an if statement checking if the interrupt is from the hardware timer (which_dev == 2). I added p->cputime++; (and myproc()->cputime++;) right before yield() so that every time the timer goes off while this process is running, we give it credit for one tick.

Task 4. (10 pts) Implement a wait2() system call that waits for a child to exit and returns the child's status and rusage.

I had to connect the new wait2() system call through all the layers of xv6:

- **kernel/pstat.h:** I created this new file and added the struct rusage definition containing uint cputime;
- **user/user.h:** Added struct rusage; and the user-level prototype int wait2(int*, struct rusage*); so user programs know how to call it.
- **user/usys.pl:** Added entry("wait2"); so the script can make the assembly stub that jumps into the kernel.
- **kernel/syscall.h:** Added #define SYS_wait2 23. I chose 23 because 22 was already taken by SYS_sync in my repo.
- **kernel/syscall.c:** Added extern uint64 sys_wait2(void); and added [SYS_wait2] = sys_wait2, to the syscall table so the kernel knows which function to call.
- **kernel/defs.h:** Added int kwait2(uint64, uint64); under the proc.c section so sysproc.c can see it.
- **kernel/sysproc.c:** Wrote sys_wait2() to read the two memory addresses passed in from the user using argaddr() and pass them along to kwait2().
- **kernel/proc.c:** Included pstat.h at the top and wrote kwait2(). It looks through the process table for an exited child process (a ZOMBIE), copies its exit status and its cputime back to the parent's memory using copyout(), cleans up the child process with freeproc(), and returns its PID.

Task 5. (10 pts) Implement a time command that runs the command given to it as an argument and outputs elapsed time, CPU time, and %CPU used.

I wrote user/time.c and added it to UPROGS in the Makefile. It saves the starting tick from uptime(), uses fork() and exec() to run the command, and uses wait2(&status, &ru) to wait for the child and grab its cputime. It takes the ending uptime(), calculates how many ticks passed, figures out the CPU percentage with (cpu * 100) / elapsed, and prints out the results.

Output:

```
$ time sleep 10
```

```
elapsed time: 10 ticks, cpu time: 0 ticks, 0% CPU
```

```
$ time matmul
```

```
Time: 40 ticks
```

```
elapsed time: 41 ticks, cpu time: 41 ticks, 100% CPU
```

Task 6. (5 pts) Summarize what you learned from doing this assignment. Describe any difficulties you faced and if/how you overcame them.

What I Learned

- **How user programs talk to the kernel:** Before this lab, system calls felt like magic black boxes. Now I understand how a function call in user space goes through an assembly stub, gets matched to a number in a table, and jumps into kernel functions.
- **How an OS measures time:** I learned that an OS doesn't have an ordinary stopwatch running; it counts hardware timer ticks. Every time an interrupt happens, the OS gets a chance to pause a process and add to its run count.
- **Memory isolation:** Because user programs and the kernel have different memory spaces, the kernel cannot just write to user variables directly. It has to use special functions like `copyout()` to safely place data into user memory.

Difficulties I Faced & How I Overcame Them

- **Syscall Number Collision:** The assignment handout said to use number 22, but my code already had a `SYS_sync` call assigned to 22. If I kept 22, they would overwrite each other. I fixed it by giving `SYS_wait2` number 23 instead.
- **Compiler Errors with `copyout()`:** When I first wrote `kwait2()`, the compiler gave me errors saying I had the wrong number of arguments for `copyout()`. I checked the existing `kwait()` function in `kernel/proc.c` and saw that my version of xv6 takes 5 arguments (`pagetable, sz, addr, src, len`) instead of 4. Updating my code to pass all 5 arguments fixed the error.
- **Missing Braces in `kerneltrap()`:** When I added `myproc()->cpuctime++` in `kernel/trap.c`, I accidentally messed up the curly braces around the if check. Because xv6 treats warnings as errors, GCC stopped the build because of misleading indentation. Wrapping both the counter increment and `yield()` inside curly braces `{ ... }` solved it.

Extra Credit (5 points). Discuss limitations of our time command. (Hint: For one limitation, consider what would happen if the command that is being timed forks child processes).

1. **Commands that fork their own children:** Our time program only calls `wait2()` on its direct child. If the program we are timing decides to use `fork()` to make grandchild processes, our time command will never see their CPU time. Their ticks are completely missed from our calculation.
2. **Timer Ticks are Coarse:** In xv6, timer interrupts happen about 100 times every second (every 10 ms). If a program is very fast and finishes before a single timer tick happens, its CPU time will show up as 0 ticks even though it actually did work.
3. **Sampling Luck / Inaccuracy:** The kernel only counts a tick if the process happens to be actively running right when the timer interrupt goes off. A process could run for 9 milliseconds, yield the CPU right before the tick, and get charged 0 ticks. Another process could run for just 1 millisecond right as the tick happens and get charged a whole tick.
4. **Kernel Overhead is Ignored:** Time spent switching between processes or running internal scheduler loops isn't counted toward the user program, which can make the wall-clock time and CPU time slightly off from reality.

In compliance with the course academic integrity policy regarding AI assistance:

- **Tool Used:** Google Gemini.
- **How It Was Used:**
 1. I used Gemini as a tutor to help me understand compiler errors, specifically why `copyout()` had too few arguments and why `kerneltrap()` failed with misleading indentation errors.
 2. It helped explain how xv6 connects system calls across `usys.pl`, `syscall.c`, and `sysproc.c` so I could trace how arguments get passed from user space into the kernel.
 3. It helped me structure my thoughts for this report.
- **Verification:** I wrote, edited, tested, and ran all the code and commands myself on my Linode virtual machine.